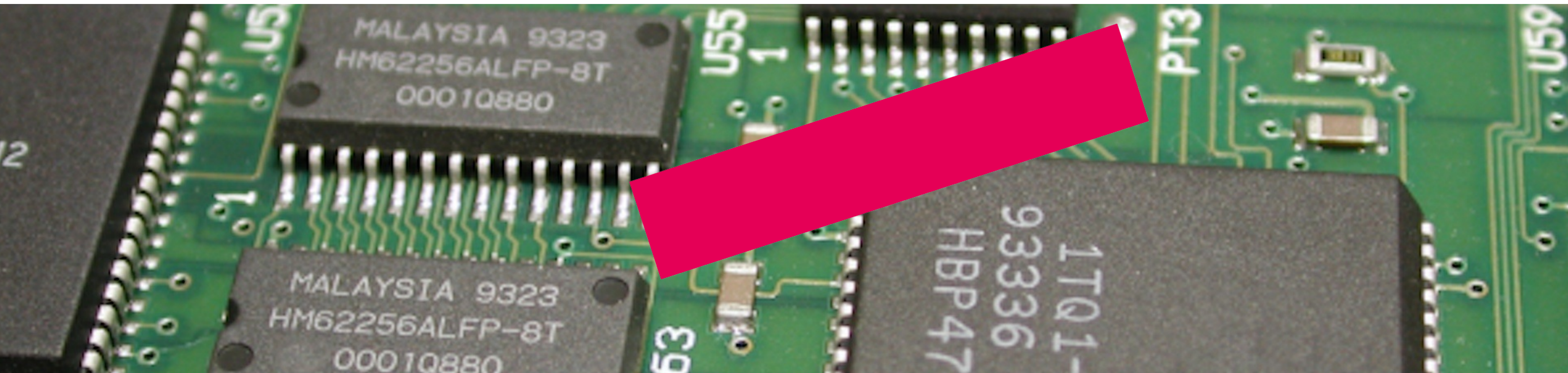


Prog 5 - 4. Interfaces and abstract classes



Electrical Engineering / Embedded Systems
Faculty of Engineering

Johan.Korten@han.nl

To begin with...

Attendance

Schedule (exact info see #00 and roster at insite.han.nl)

Week (Thu)	Design principle	C++	UML	Lab (Fri)
Step 1 - 3 Sep	Single Responsibility	Scope, namespaces, string	Use cases / class diagram	4 Sep: Git
Step 2 - 10 Sep	Open-Closed Principle	Constructors, iterators, lambdas	Inheritance / generalization	11 Sep: Requirements
Step 3 - 17 Sep	Liskov Substitution Principle	Lists, inline functions, default params	Activity diagrams	18 Sep: sensor library 1: core, Bus, Arduino
Step 4 - 24 Sep	Interface Segregation Principle	Interfaces and abstract classes	Dependencies	25 Sep: sensor library 2: Raspberry Pi, Bus / Clock split
Step 5 - 1 Oct	Dependency Inversion Principle	Threads, callbacks	Sequence diagrams	2 Oct: sensor library 3: sampler thread, callback, MQTT
Step 6 - 8 Oct	Coupling and cohesion	Polymorphism	Composition, packages	9 Oct: sensor library 4: polymorphism, UML design
Step 7 - 15 Oct	Embedded SOLID	Lifecycle: constructors, destructors, const	-	16 Oct: sensor library 5: RAIL, host unit tests, refactor

Note: subject to changes as we go...

Interfaces versus Abstract classes

Two concepts:

- Use an ***interface*** when you want to define a common capability or behavior that different, unrelated classes should support.
- Use an ***abstract class*** when your subclasses share code, state, or a tight hierarchical relationship.

But...

S - Single Responsibility Principle (SRP): A class should have only one reason to change, meaning it should perform only one clear job or function.

O - Open/Closed Principle (OCP): Software entities should be open for extension, but closed for modification. You should add new features without changing existing, working code.

L - Liskov Substitution Principle (LSP): Child classes must be able to replace parent classes without breaking the application.

I - Interface Segregation Principle (ISP): Clients should not depend on methods they do not use. Split large, general interfaces into smaller, specific ones.

D - Dependency Inversion Principle (DIP): High-level modules should depend on abstractions (like interfaces), not on concrete low-level details.

Pure virtual in three rules

1. `virtual void f() = 0;` declares a function the base class does not implement. The `= 0` is syntax, not a value.
2. A class with at least one pure virtual function is abstract: you cannot create an instance of it, only of a subclass that implements every pure virtual function.
3. C++ has no interface keyword. An interface is an abstract class with only pure virtual functions and no data. Give it a virtual destructor.

That is all the language offers. Everything else today is about how you use it.

Show us: your Arduino library from last week

Please let one of you shares the screen with last week's library.

We look for three things:

1. Where do the Wire calls live? In the sensor class, in a helper, in the sketch?
2. What does the constructor receive? Nothing, a TwoWire*, an I2CHelper? (That is dependency injection: you were steered towards it.)
3. What has to change to run this on a second I2C bus (Wire1), or on a Raspberry Pi, where there is no Wire at all?

Keep your answer to 3; we come back to it in ten minutes.

Many libraries do it this way: Wire, directly

```
class Bme280 {
public:
    bool begin() {
        Wire.begin();                // <- who owns this bus?
        Wire.beginTransmission(_addr);
        Wire.write(0xD0);            // chip id register
        Wire.endTransmission(false);
        Wire.requestFrom(_addr, (uint8_t)1);
        return Wire.read() == 0x60;
    }
    float readTemperature() {
        Wire.beginTransmission(_addr); Wire.write(0xFA); Wire.endTransmission(false);
        Wire.requestFrom(_addr, (uint8_t)3);
        ...
    }
private:
    uint8_t _addr = 0x76;
};

// Works. On an Arduino, on the default bus only: Wire1 or a second sensor on another
// bus means rewriting every method. And Wire.begin() inside the sensor: a device on the
// bus initializes the bus. Two sensors, two begin()s; the clock speed the sketch set is gone.
// Question of the day: what has to change to run this on a Raspberry Pi?
```

The likely shape after Lab 3: injected, but concrete

```
class Bme280 {
public:
    Bme280(uint8_t addr = 0x76, I2CHelper i2c = I2CHelper(&Wire))
        : _addr(addr), _i2c(i2c) {}

    bool begin() {
        return _i2c.readRegister<uint8_t>(_addr, 0xD0) == 0x60;    // chip id
    }
    float readTemperature() {
        uint8_t raw[3];
        _i2c.readRegisters(_addr, 0xFA, raw, 3);
        ...
    }
private:
    uint8_t _addr;
    I2CHelper _i2c;    // wraps a TwoWire*
};
```

// Dependency injection: good. The sensor no longer calls Wire itself.
// But what is injected is a concrete Arduino type. Inject an abstraction instead: next slide.

Inject an abstraction instead: the Bus interface

```
class Bus {
public:
    virtual ~Bus() = default;

    // Read `len` bytes starting at register `reg`.
    virtual bool read(uint8_t reg, uint8_t* data, size_t len) = 0;

    // Write `len` bytes starting at register `reg`.
    virtual bool write(uint8_t reg, const uint8_t* data, size_t len) = 0;

    // Block for at least `us` microseconds.
    virtual void delayUs(uint32_t us) = 0;
};

// Only <stdint> and <stddef>. No Wire.h, no Linux, no Arduino. No begin(): the sensor
// gets a Bus& that is already up. Whoever owns the bus (setup() / main()) begins it and
// decides what it is: Wire, Wire1, /dev/i2c-1, a mock.
// If yours injects an I2CHelper: let I2CHelper implement Bus. That is step 1 of Friday's lab.
```

An interface is a contract: platform 1, Arduino

```
bool ArduinoI2cBus::read(uint8_t reg, uint8_t* data, size_t len) {  
    wire_.beginTransmission(address_);  
    wire_.write(reg);  
    if (wire_.endTransmission(false) != 0) return false;    // repeated start  
    if (wire_.requestFrom(address_, len) != len) return false;  
    for (size_t i = 0; i < len; ++i) data[i] = wire_.read();  
    return true;  
}
```

```
// The only file in the Arduino build that includes <Wire.h>.
```

Same contract: platform 2, Raspberry Pi

```
bool rpi::LinuxI2cBus::read(uint8_t reg, uint8_t* data, size_t len) {
    i2c_msg msgs[2] = {
        { address_, 0, 1, &reg }, // write register
        { address_, I2C_M_RD, uint16_t(len), data } // repeated start, read
    };
    i2c_rdwr_ioctl_data xfer{ msgs, 2 };
    return ::ioctl(fd_, I2C_RDWR, &xfer) >= 0;
}
```

```
// The only file in the Pi build that includes <linux/i2c-dev.h>.
// Bme280 did not change. Not one line.
```

Live demo: the same library on the Pi

On the Pi:

```
i2cdetect -y 1                # sensor at 0x76 or 0x77
cd Prog5/Code/week4/rpi_platform
cmake -B build && cmake --build build
./build/rpi_smoke_test
```

On the laptop:

```
cmake -B build && cmake --build build
-> core and tests build, rpi_smoke_test does not (Linux only)
```

That difference is the layering:

```
bme280 (core) -> rpi_platform (Linux) -> rpi_smoke_test (app)
```

Abstract classes vs. Interfaces: UML

Dog is an **abstract class**: it has data (name, legs) and real code (eat, sleep). Only barks() is left open. A Chihuahua inherits half a body and fills in one gap.

Pet is an interface: no data, no code, only promises (play, cuddle). A cat, a goldfish and a PetDog can all play that role; they share nothing else.

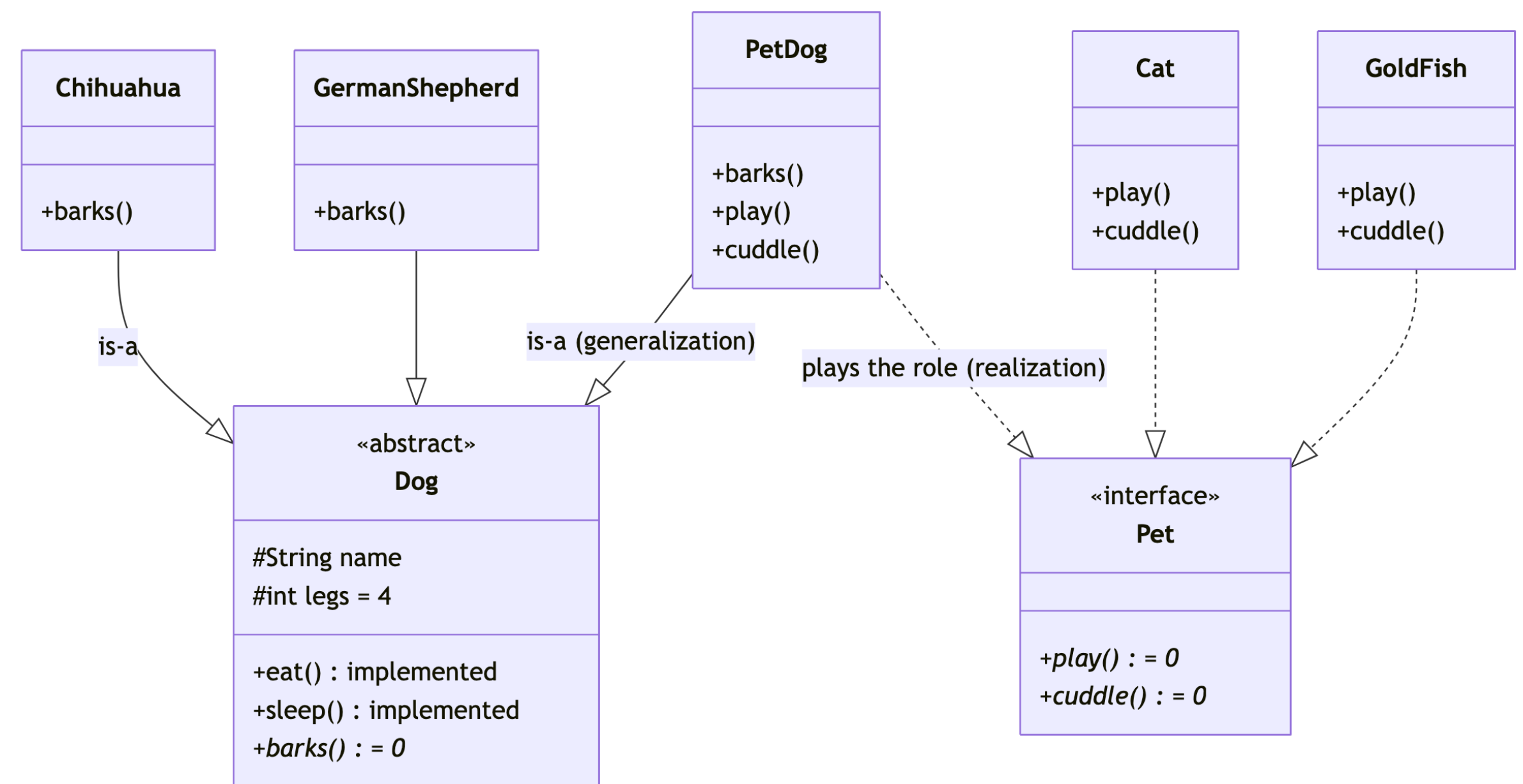
Dog gives you half a body. Pet gives you a promise.

Solid arrow:

is-a (generalization).

Dashed arrow:

plays the role (realization).



Abstract classes vs. Interfaces: UML, on the sensor library

Interface: a *role*. Only pure virtual functions, no data.

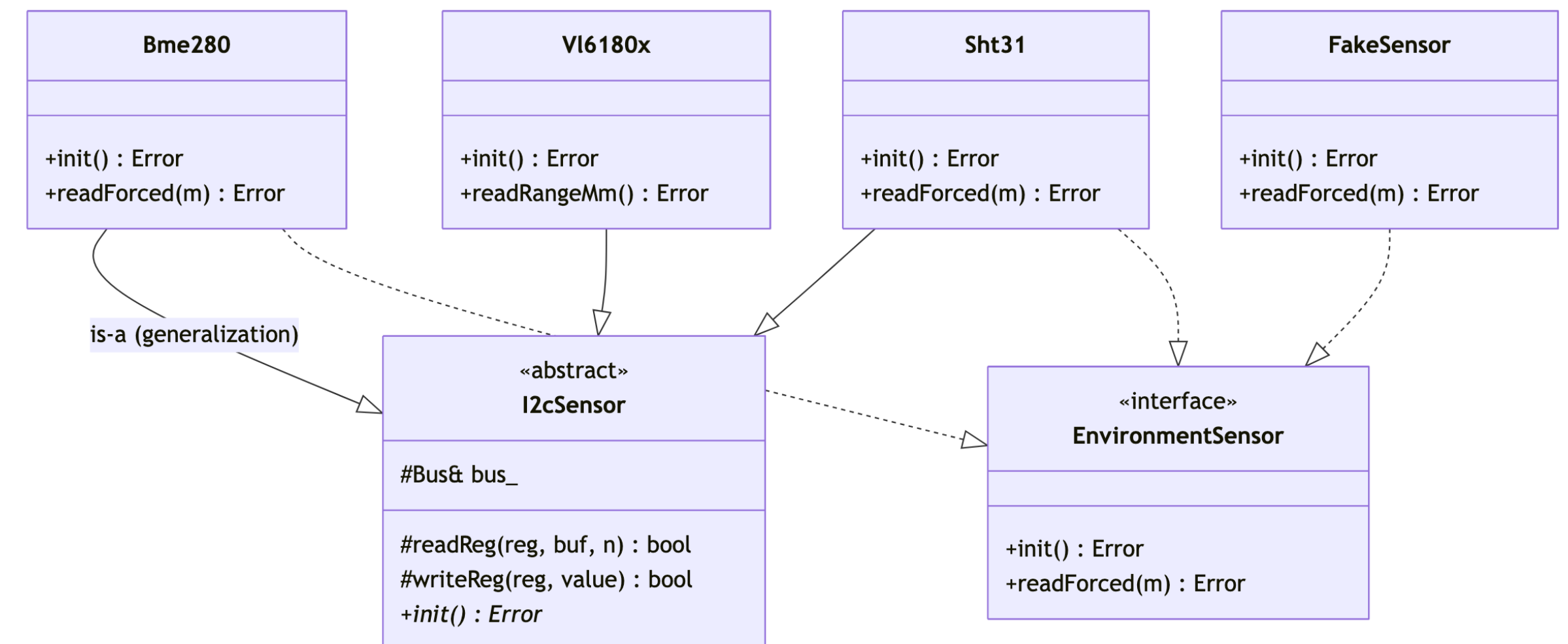
EnvironmentSensor: "can be asked for a measurement".

FakeSensor implements it without any I2C.

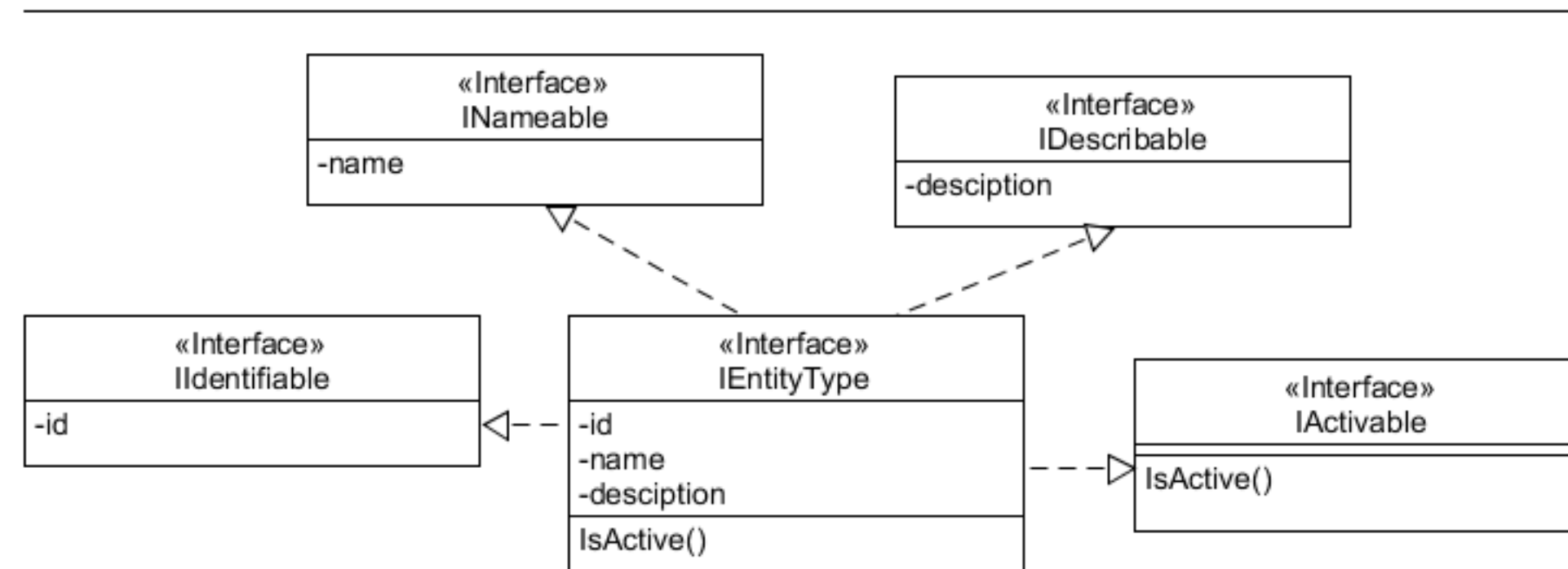
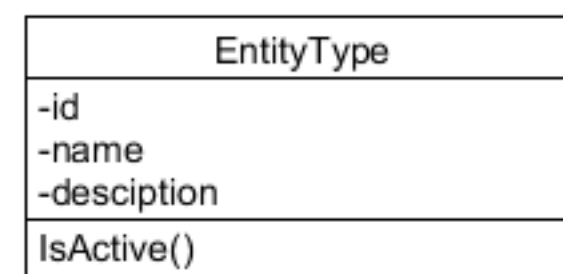
Abstract class: shared implementation plus at least one pure virtual. I2cSensor holds the Bus& and the readReg / writeReg helpers and leaves init() to the subclass. Bme280 is-a I2cSensor (generalization, solid arrow) and implements EnvironmentSensor (realization, dashed arrow).

For the sensor library we use *interfaces*:
every seam we want to swap, mock or run on another platform
(Bus, Clock, EnvironmentSensor).

Shared helpers go into a small class you compose, not into a
base class you inherit.



SOLID: Interface Segregation



"No client should be forced to depend on methods it does not use."

- Robert C. Martin

Make interfaces as slender as possible.

An interface that carries operations its clients never call is a fat interface: every implementation pays for it and every test double has to fake it. Next slide: we have one.

<https://claudiorivera.net/2018/02/04/isp-interface-segregation-principle/>

Interface Segregation: the fat sensor interface everyone writes first

```
class Sensor {                                // one interface for every sensor we own
public:
    virtual float readTemperature() = 0;
    virtual float readPressure()    = 0;
    virtual float readHumidity()    = 0;
    virtual float readDistance()    = 0;        // for the VL6180X from Lab 3
};

float Vl6180x::readHumidity() { return NAN; }    // "not applicable"
float Vl6180x::readPressure() { return NAN; }
float Bme280::readDistance()  { return -1.0f; } // "not applicable", differently

// Every implementer pays for methods its clients never call.
// Every caller has to know which NaN means what.
// Cut along the clients, not along the devices:
//   EnvironmentSensor { init(); readForced(Measurement&); }    what the station asks
//   RangeSensor       { init(); readRangeMm(uint16_t&); }     what a distance loop asks
// Bme280 implements the first, Vl6180x the second, nobody returns NAN.
// Next slide: the same smell, three sizes smaller, in your own Bus.
```

Interface Segregation, spotted: Bus::delayUs()

```
class Bus {                                // one interface, two jobs
    virtual bool read(uint8_t reg, uint8_t* d, size_t n) = 0;           // moving bytes
    virtual bool write(uint8_t reg, const uint8_t* d, size_t n) = 0;    // moving bytes
    virtual void delayUs(uint32_t us) = 0;                               // waiting. Why here?
};

// Who needs delayUs()? One class: Bme280, which waits a few ms after starting a measurement.
// Who has to implement it? Every class that implements Bus:
void ArduinoI2cBus::delayUs(uint32_t us) { delayMicroseconds(us); }    // nothing to do with I2C
void LinuxI2cBus::delayUs(uint32_t us)  { usleep(us); }                // nothing to do with I2C
void MockBus::delayUs(uint32_t us)      { delayedUs_ += us; }          // a fake for the fake's sake
// A future SpiBus: copy the sleep line a fourth time.
// The test "does readForced() wait long enough?" ends up asking MockBus, a class about registers.

// "No client should be forced to depend on methods it does not use."
// Two kinds of clients here: the classes that implement Bus (forced to provide a delay)
// and Bme280 (forced to get time from a bus). Bytes and time do not belong in one contract.
// Split it off: next slide.
```

The fix: cut along the clients. Bus and Clock

```
class Bus {
public:
    virtual ~Bus() = default;
    virtual bool read(uint8_t reg, uint8_t* data, size_t len) = 0;
    virtual bool write(uint8_t reg, const uint8_t* data, size_t len) = 0;
};

class Clock {
public:
    virtual ~Clock() = default;
    virtual void delayUs(uint32_t us) = 0;
};

class Bme280 {
public:
    Bme280(Bus& bus, Clock& clock);    // two small contracts, not one fat one
    ...
};

// ArduinoClock, LinuxClock, FakeClock (tests: only counts the microseconds)
```

UML: dependencies. No arrow leaves the core

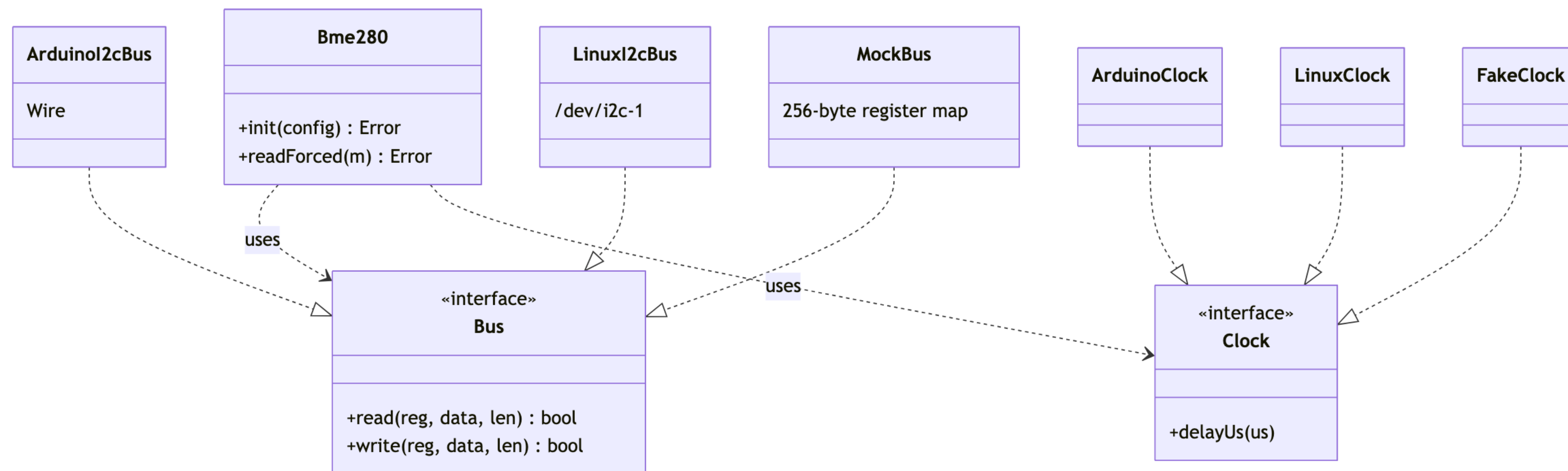
Realization (dashed, hollow head): implements

Dependency (dashed, open head): uses

Rule for your class diagram: not one arrow leaves the core towards a platform.

Try it: draw Wire and ioctl into this picture. They can only hang under the platform classes.

If you cannot draw it that way, the code has the problem, not the diagram.



Handing in: one repository, one tag per assignment

Assignments 3 to 8 grow the same library in the same repository. A tag marks the commit you hand in, so we look at exactly that version while you keep working.

v0.3-arduino	assignment 3	core + Arduino
v0.4-pi	assignment 4	Raspberry Pi, Bus / Clock
v0.5-mqtt	assignment 5	sampler, callback, MQTT
v0.6-design	assignment 6	UML design + polymorphism
v1.0	assignments 7 + 8	tests, refactor

You, after the last commit of the week:

```
git tag v0.4-pi && git push origin v0.4-pi
```

Us, when reviewing:

```
git checkout v0.4-pi
```

Forgot to tag? Tag the deadline commit afterwards. Wrong commit tagged? `git tag -d`, retag, push with `--force`. Assignment 8 reviews your own code at v0.6-design, so the tags are for you too.

Assignment 4: same library, second platform (deadline Thu 1 Oct, tag v0.4-pi)

1. Build the core on the Pi with CMake: three targets (Bosch C, wrapper, tests). Prove it builds on the Pi and on the laptop.
2. LinuxI2cBus: open in the constructor, close in the destructor, non-copyable.
3. Split Bus / Clock (ISP). ArduinoClock, LinuxClock, FakeClock. Write in the README what got simpler.
4. Smoke test on the Pi: one measurement per second, clean exit on Ctrl-C.
5. Class diagram with realizations and dependencies. No arrow leaves the core.

Layout: platform/arduino, platform/linux, examples/arduino, examples/rpi.

Full hand-out, starter code and tags: github.com/AEAEmbedded/ESE_PROG -> Prog5/Assignments

Links

Code of this lecture

- github.com/AEAEmbedded/ESE_PROG -> Prog5/Code/week2 (core, Bus, MockBus), week4/rpi_platform (LinuxI2cBus)

The sensor and its driver

- Bosch BME280 SensorAPI (C, BSD-3): github.com/boschsensortec/BME280_SensorAPI
- BME280 datasheet: bosch-sensortec.com/products/environmental-sensors/humidity-sensors-bme280/

The two buses

- Arduino Wire: docs.arduino.cc/language-reference/en/functions/communication/wire/
- Linux i2c-dev (I2C_RDWR): kernel.org/doc/html/latest/i2c/dev-interface.html

Links

Interfaces in C++

- C++ Core Guidelines, C.hier: C.121 pure abstract interfaces, C.127 virtual destructors, C.129 interface vs implementation inheritance
- isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines
- Robert C. Martin, Clean Architecture (2018), ch. 10: Interface Segregation Principle
- Martin Fowler, Inversion of Control Containers and the Dependency Injection pattern: martinfowler.com/articles/injection.html

Diagrams and tests

- Mermaid class diagrams: mermaid.js.org/syntax/classDiagram.html
- doctest (used by the starter's host tests): github.com/doctest/doctest

Try it yourself...

Prompt: “What is the best way to use interfaces in C++?”

<https://copilot.microsoft.com/>

<https://gemini.google.com/>

<https://claude.ai/>

<https://chatgpt.com/>

Which one do you prefer?!

“

Any questions?